Laya Healthcare

Dataset Validation

Overview Guide — PDF 1 of 2

This document gives you a complete but concise understanding of the synthetic dataset and all 14 validation checks. Every check is explained in plain English — what it does, what the result means, and what you need to do before training. For full code walkthroughs see **PDF 2: Deep Dive Guide**.

412,042 Total rows	7 Tables	14 Checks run	11 / 3 / 0 Pass /Note/Fail	20.5% Positive label

01 What Is This Dataset?

Seven interrelated CSV files simulating 6 months of Laya Healthcare member activity (June–November 2025). The goal is to train a model predicting whether a user will call support within 48 hours after a claim snapshot — enabling proactive interventions.

Table	Rows	What it contains
users.csv	2,000	Member demographics, plan types, archetypes
treatment_master.csv	7	Reference: claim types, SLAs, target rates
claims.csv	3,799	Claims with flags, amounts, status
claim_status_history.csv	12,251	All status changes per claim over time
app_logs.csv	361,903	Every user action in the app
support_calls.csv	5,477	Support calls linked back to claims
feature_snapshots.csv	30,710	ML training table — one row per claim per day

feature_snapshots.csv is the actual ML input. All other tables are the raw source.

02 The 14 Validation Checks — At a Glance

#	Check	Result	What was confirmed
1	**Row Counts**	PASS	All 7 tables populated, no empty tables
2	**Null Analysis**	PASS	Nulls only in expected columns
3	**FK Integrity**	PASS	0 orphan rows — all 4 checks clean
4	**Chronological Order**	PASS	0 status timestamps before submission
5	**User Distributions**	PASS	Age, archetype, plan within +/-3pp of YAML

6	**Claim Type Shares**	PASS	All 7 types within +/-2.5pp of YAML
7	**Claim Amounts**	PASS	Medians within 15% of YAML targets
8	**Flag Rates**	PASS	Missing doc / adj / rejection rates correct
9	**Label Distribution**	NOTE	20.5% positive — 3.9:1 imbalance (expected)
10	**Escalation Logic**	PASS	Distress > Anxious, 75+ highest, rejected highest
11	**Noise Injection**	PASS	Variants, nulls and outliers confirmed present
12	**Feature Stats**	PASS	All features have sensible ranges and spread
13	**Channel Distribution**	NOTE	paper_post +5.6pp — age-skew effect, not a bug
14	**Maternity Miss Doc Rate**	NOTE	21% vs 14% target — small sample n=205

03 What Each Check Does — Plain English

1 · Row Counts	Prints row and column counts for all 7 tables. Confirms nothing is empty. app_logs should dominate at ~360K rows because every user action creates a log entry.
2 · Null Analysis	Scans every column for missing values and classifies them as Expected (structural nulls that exist by design) or Investigate (gaps that could corrupt features). original_claim_id being 96% null is fine — only resubmissions have a parent. claim_amount being null would be a serious problem.
3 · FK Integrity	Checks that every ID in a child table exists in the parent table. Claims must belong to real users. Status rows must reference real claims. Even one orphan row means feature engineering will silently fail.
4 · Chronological Order	A claim must be submitted before its status can be updated. This catches timestamp bugs from the 2% DST glitch injection. Ireland's clock change in late October can create small negative time gaps.
5 · User Distributions	Compares actual percentages for age group, plan type, and behaviour archetype against YAML targets. A +/-3pp drift is normal with random sampling of 2,000 users. It would be suspicious if every column was exactly right.
6 · Claim Type Shares	Verifies GP, Dental, Surgical etc. appear in the right proportions. Also confirms noise variants (dental, DENTAL, Physio) exist at ~3%. Finding zero variants would mean the noise injection failed.
7 · Claim Amounts	Checks that money values are calibrated per claim type. Uses median not mean because the 1% outlier injection (x10 multiplier) skews the mean upward. Tolerance is 15% of target. Surgical mean > median is expected — log-normal distribution.
8 · Flag Rates	Verifies missing document, adjudicator, and rejection rates per claim type. These flags directly drive escalation probability so calibration matters for label quality. Maternity missing doc rate is +7pp above target — small sample effect, acceptable.
9 · Label Distribution	Counts escalation (1) vs no-escalation (0) in the ML training table. 20.5% positive with 3.9:1 imbalance. Expected and realistic. Never evaluate on accuracy — use PR-AUC or F1. Set scale_pos_weight=4 in XGBoost.
10 · Escalation Logic	The most important check. Confirms the right users are escalating: Distress > Anxious > Passive > Engaged, 75+ age group highest, rejected claims highest. If these orderings fail, the training labels encode the wrong patterns.
11 · Noise Injection	Confirms all four noise types were injected at correct rates: 3% claim type variants, 4% region name variants, 1% outliers, 8% session duration nulls. These test your preprocessing pipeline's ability to handle real-world messy data.
12 · Feature Stats	Descriptive statistics for all 12 numeric features. Checks for impossible values and confirms spreads make sense. Overlapping histograms preview which features separate escalation from non-escalation — these will top your feature importance ranking.

04 The Three Notes — What They Mean and What To Do

Class Imbalance — 18.3:1

18.3 no-escalation rows for every 1 escalation row. Expected for a rare-event predictor.

- Do NOT use accuracy as your evaluation metric
- Use class_weight='balanced' in scikit-learn models
- Use scale_pos_weight=4 in XGBoost / LightGBM
- Evaluate on: PR-AUC (primary), F1 at threshold 0.3, Recall at precision > 0.5
- Apply SMOTE only on the training fold — never on validation or test data

paper_post Channel +5.6pp over target

23.6% actual vs 18% YAML target. This is the age-channel interaction working correctly.

- 60+ and 75+ users are correctly skewed toward paper submission as configured
- The marginal distribution drifts because those age bands are large
- No action required — the model sees the correct behaviour
- If strict marginal fidelity is needed: reduce paper weight for under-60 users in config

Maternity Missing Doc Rate +7pp

21% actual vs 14% YAML target. Small sample with compounding interaction.

- Caused by small sample size (n=205) — larger variance is expected at this scale
- Adjudicator-to-missing-doc interaction compounds slightly more for Maternity
- Acceptable for training data — will not materially affect the model
- Monitor when real Laya data arrives and compare actual Maternity doc failure rates

05 Next Steps Before Model Training

Preprocessing

- Normalise treatment_type and region variants — lowercase + regex map to canonical names
- Winsorise claim_amount and relative_claim_cost at the 99th percentile
- Impute days_since_resubmission with 0 when resubmission_flag = False
- Impute session_duration nulls with median per event_type bucket

Feature Engineering

- log1p(claim_amount), log1p(behavior_acceleration), log1p(relative_claim_cost) — all right-skewed
- stress_score = missing_doc + rejected + adj_flag + resubmit (weighted composite risk)
- channel_risk_flag = 1 if paper_post — paper users cannot self-serve status checks
- age_paper_interaction = 1 if age 75+ AND paper_post — highest risk combination

Modelling

- Baseline first: Logistic Regression with class_weight='balanced'
- Primary: XGBoost or LightGBM with scale_pos_weight=4, use early_stopping_rounds
- Metric: PR-AUC as primary, F1 at threshold 0.3 as secondary — never accuracy
- Train/test split MUST be time-based: train Jun-Sep, test Oct-Nov (not random)

For line-by-line code explanations, the reasoning behind every design decision, and full breakdowns of how each check works — see PDF 2: Deep Dive Guide.